

Как мы вообще до этого дошли?

Эволюция инженерной разработки ПО

1950-е – 1960-е

Вычислительные центры

Большие машины, перфокарты,
пакетный режим



1970-е – 1980-е

Персональные компьютеры

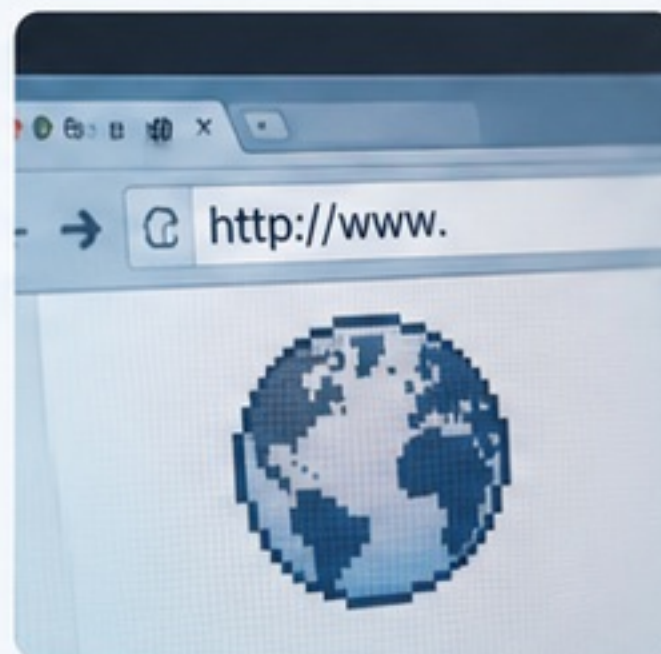
Появление ПК и массового
программирования



1990-е – 2000-е

Интернет и веб

Глобальные системы,
распределённые команды



2010-е

Облако и DevOps

Автоматизация,
непрерывные поставки



2020-е –...

AI и новые вызовы

Сложные системы, больше
автоматизации, выше требования
к надёжности и безопасности



Каждый этап появлялся как ответ на новые задачи и ограничения.

Современные инженерные практики — результат десятилетний опыта.

Во времена динозавров

1950-е — начало 1960-х · Вычислительный центр

Компьютер — большая дорогая общая машина. Программист не работает с ней напрямую: он готовит задание и передаёт его оператору.

Программист

Пишет программу и подготавливает задание.



Перфокарты

Физический носитель программы и данных.



Оператор

Управляет машиной и запускает подготовленные задания.



Очередь заданий

Машина выполняет задания последовательно (пакетный режим).



Mainframe

Дорогой централизованный вычислительный ресурс.



Результат

Обычно распечатка или другой физический вывод.



Если есть ошибка — весь цикл повторяется.

Изменил одну строку кода?

Изменение



Подготовка карт



Очередь



Запуск



Результат

Обратная связь могла занимать очень много времени.

Главное ограничение эпохи — время между изменением кода и результатом.

Компьютер начинает отвечать человеку

1960-е — 1970-е · Terminals, time-sharing и Unix

Пользователь получает терминал и может взаимодействовать с общей машиной напрямую. Вместо длительного цикла через оператора появляется интерактивная работа.



Новый цикл разработки

Ввёл команду → Получил результат → Исправил → Запустил снова

Обратная связь теперь занимает секунды, а не часы или дни.

1969 — Unix

Идеи, которые определили современную разработку.



Файлы

Программы и данные организованы через filesystem.



Процессы

Запущенная программа становится отдельным процессом.



Терминал и shell

Управление системой через команды.



Небольшие утилиты

Простые программы можно соединять между собой.

```
$ ls
a.out  hello.c  docs/
$ cc hello.c
$ ./a.out
Hello, world!
$ _
```

Простой пример работы в Unix-подобной системе.

Обратная связь становится интерактивной: разработчик начинает работать с компьютером напрямую.

Терминал, файловая система, процессы и командная строка, появившиеся в эту эпоху, переживут десятилетия.

Кодом нужно управлять

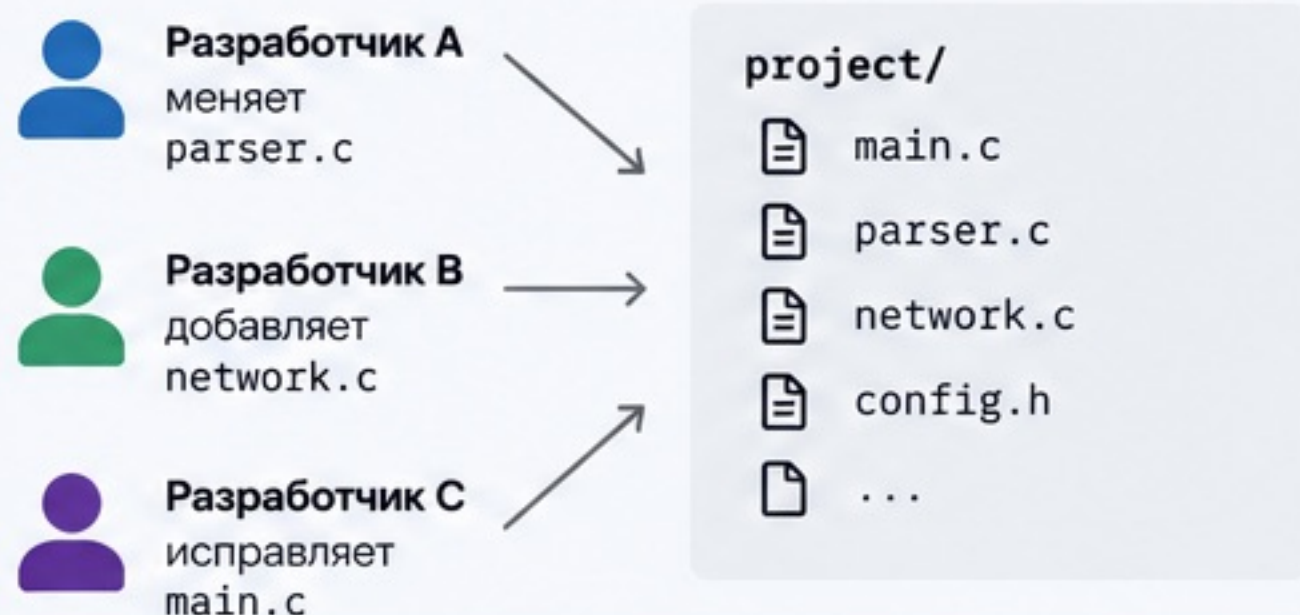
1970-е — 1980-е · Первые системы контроля версий и автоматизация сборки

Программ становится больше, они сложнее, и над ними работают несколько человек.

Просто хранить файлы с названиями вроде «project_final_v2_really_final» больше не работает.

1. Кто и что изменил?

Несколько разработчиков работают над одним проектом.



Возникают вопросы:

- Какая версия актуальная?
- Что изменилось?
- Кто это изменил?
- Как вернуться к предыдущей версии?

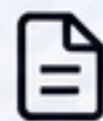
Первые системы контроля версий:

SCCS (1972) → RCS (1982) → CVS (конец 1980-х)

Фиксировали историю изменений файлов и позволяли работать с разными версиями кода.

Проект становится системой

Нужно сохранять не только код, но и способ работы с ним.



Исходники
Код программы



История изменений
Кто, что и когда изменил



Правила сборки
Как из исходников получить работающее приложение

2. Как собрать программу одинаково два раза?

Сборка сложного проекта — это не одна команда.

- 1 Компилировать main.c
- 2 Компилировать parser.c
- 3 Компилировать network.c
- 4 Сгенерировать файлы
- 5 Слинковать
- 6 Скопировать конфиги
- 7 ...

Когда шагов много, возникают проблемы:

- Легко что-то забыть
- Разный результат на разных машинах
- Сложно повторить сборку
- Трудно делиться процессом с другими

Автоматизация сборки

Повторяемую последовательность действий нужно описать один раз, а не запускать вручную каждый раз.

Makefile

```
1 app: main.o parser.o
2     cc -o app main.o parser.o
3
```

make

выполняет нужные шаги в правильном порядке.

Как только код становится общим и сложным, появляются две потребности: история изменений и воспроизводимая сборка.

Git и современные pipelines — гораздо более поздние ответы на те же базовые проблемы.

Компьютер переезжает на стол разработчика

1980-е — начало 1990-х · Personal computers и коробочное ПО

Компьютер перестаёт быть только общим ресурсом. У разработчика появляется собственная машина, а программы начинают массово поставляться пользователям как отдельный продукт.

Разработка становится локальной

У разработчика есть собственный компьютер, на котором можно писать, собирать и запускать программы.



Машинное время больше не главный дефицит.

Цикл «изменил → скомпилировал → запустил → исправил» теперь занимает секунды или минуты, а не часы или дни.

ПО становится массовым продуктом

Программы распространяются на физических носителях: дискетах, CD. Появляется рынок коробочного ПО.



Выпустили ошибку?

Чтобы донести исправление до пользователей, нужно:

Исправить код



Собрать новую версию



Записать на носители



Распространить (магазины, рассылка)



Пользователь установит обновление

Как менялась вычислительная среда



1950-е
Большая общая машина



1970-е
Терминалы, общий компьютер



1980-е
Компьютер у разработчика



**Разрабатывать стало значительно быстрее.
Доставлять изменения пользователю — всё ещё дорого.**

Локальная разработка ускорилась, но выпуск и распространение ПО оставались отдельной и сложной задачей.

Программа становится сервисом

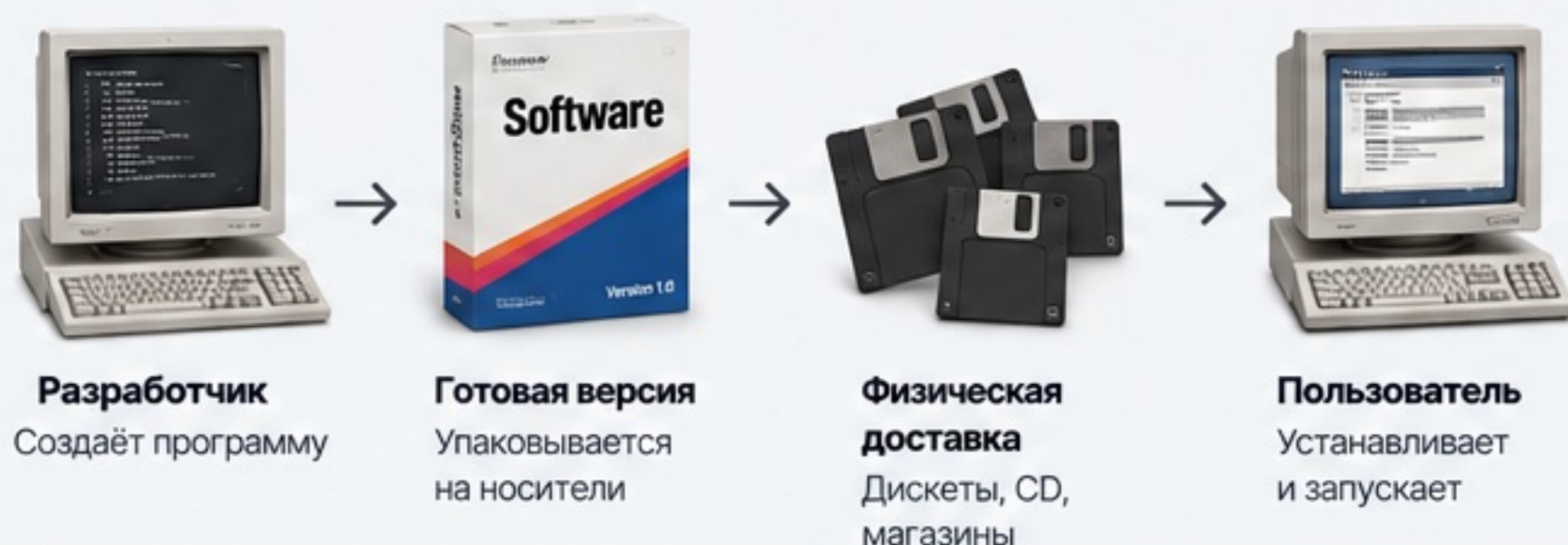
1990-е · Internet, Linux и web

Интернет связывает компьютеры между собой, а web позволяет пользователю работать с программой через браузер. Теперь обновить серверную часть можно централизованно, не доставляя новую копию программы каждому пользователю.

Раньше: коробочное ПО

1980-е — начало 1990-х

Каждый пользователь получает собственную копию программы. Чтобы выпустить обновление, его нужно снова доставить.



Нашли ошибку? Нужно собрать новую версию, записать носители и снова доставить её всем пользователям.

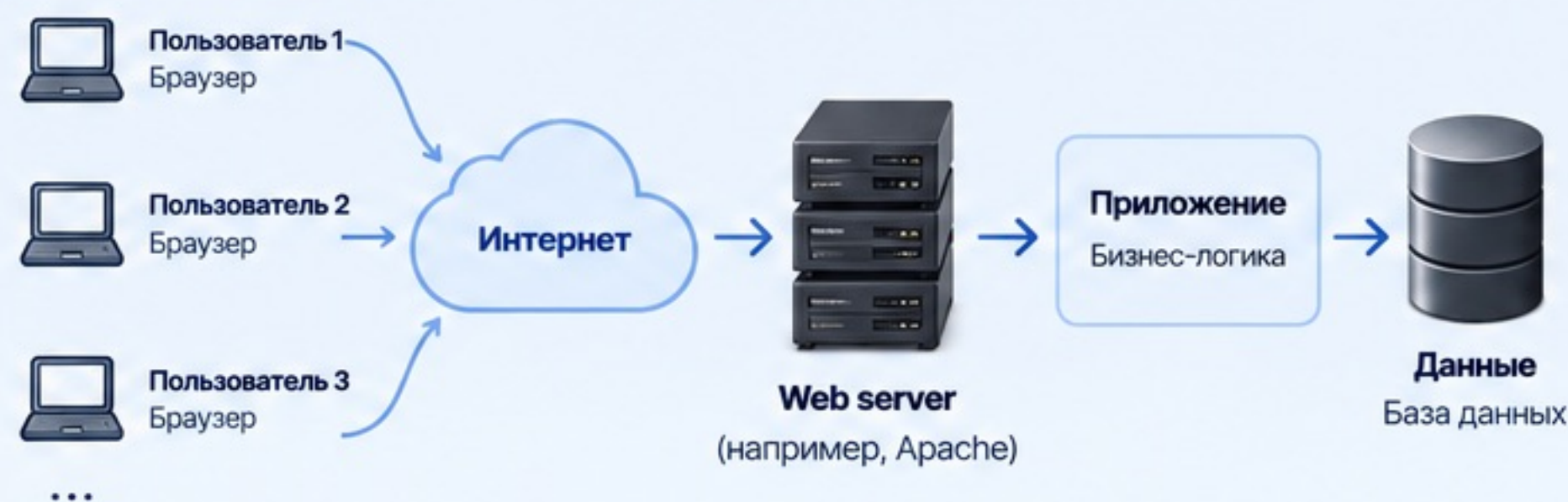
Теперь: web и сервисы

1990-е · Internet и World Wide Web

Пользователь обращается к программе через браузер. Одна серверная версия обслуживает многих пользователей.



Обновили сервер — новая версия доступна всем пользователям.



Что изменилось?

- 1 Программа работает постоянно**
Сервис должен быть доступен тогда, когда нужен пользователю.
- 2 Один экземпляр обслуживает многих**
Обновление становится централизованным. Но ошибка на сервере может затронуть сразу всех пользователей.
- 3 Разработка становится распределённой**
Интернет используется не только пользователями, но и самими разработчиками. Появляются глобальные сообщества и open source.



1991 — Linux

Интернет + open source + совместная разработка

Linux стал одним из первых крупных проектов, который создавался открыто и совместно разработчиками со всего мира через Интернет.



1995 — Apache

Веб-сервер, созданный коллективно

Разработчики из разных стран объединяли усилия через Internet и mailing lists. Apache стал одним из символов новой, сетевой модели разработки ПО.



Продукт всё чаще становится постоянно работающим сервисом, который команда может обновлять централизованно.

Цена релиза снижается — цена недоступности растёт.

Интернет изменил не только то, как пользователи используют программы, но и то, как их создают.

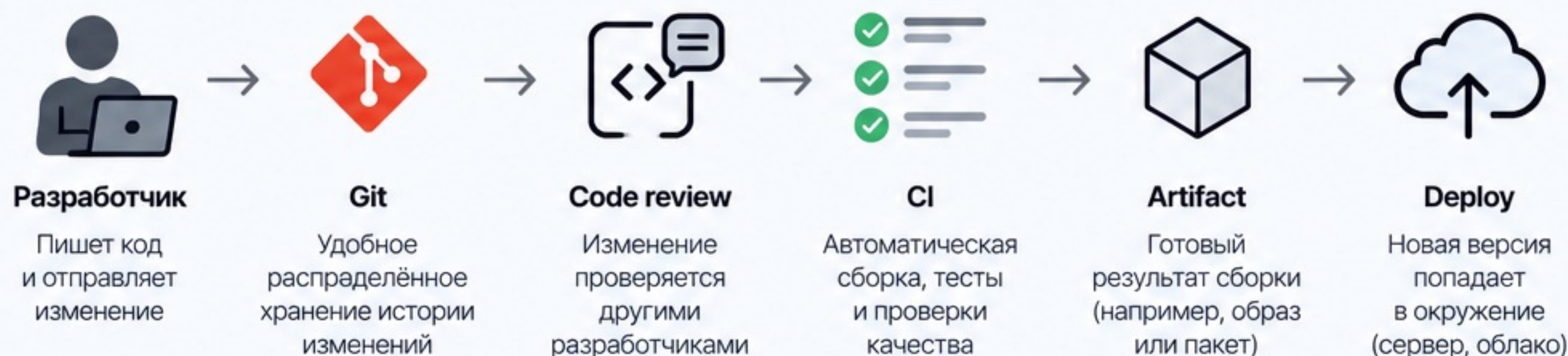
Разрабатывать нужно быстрее

2000-е · Git, CI, code review и cloud

*Small changes.
Big impact.*

Изменения становятся меньше и чаще. Ручная сборка, редкие большие релизы и отсутствие автоматизации начинают мешать скорости команды.

Новый процесс доставки изменений



Важные вехи



2005 — Git

Распределённая система контроля версий, ориентированная на скорость и удобство командной работы.



2000-е — CI становится нормой

Автоматическая сборка, тестирование и проверки начинают использоваться регулярно, а не только перед релизом.



Cloud

Инфраструктуру становится проще получать по требованию, без покупки и настройки каждого сервера вручную.

Как меняется подход к релизам

Раньше

Большие и редкие релизы

- Долго кодим изменения
- Ручная сборка и тесты
- Высокий риск ошибок
- Долгая доставка

Релиз раз в месяцы



Теперь

Маленькие и частые изменения

- Небольшие независимые изменения
- Автоматические проверки
- Быстрая доставка
- Проще понять, откатить и исправить

Релиз хоть каждый день

Пример CI-сборки

```
► Run tests
✓ Checkout code
✓ Install dependencies
✓ Run unit tests      12.4s
✓ Run lint            3.1s
✓ Build project       28.7s
✓ Build artifact      5.2s
✓ Deploy to staging   9.8s
✓ Pipeline succeeded
```

Примеры инструментов



GitHub

Git, code review, issues



GitLab

Git, CI/CD, registry



Jenkins

CI/CD



Docker

Контейнеризация



Kubernetes

Оркестрация



Облачные платформы

Инфраструктура по требованию



Разработка ускоряется не потому, что люди печатают код быстрее, а потому что всё больше рутинных шагов становится автоматизированным.

Изменения становятся меньше, чаще и предсказуемее.

Автоматизация превращает релизы из сложного и редкого события в регулярный и управляемый процесс.

Теперь проблема — масштаб

2010-е · Containers, orchestration, IaC и DevOps

Больше сервисов —
больше вопросов

Когда приложений, сервисов и окружений становится много, главный вопрос уже не «как запустить программу», а «как запускать и изменять всё это одинаково, предсказуемо и много раз».

Раньше: один сервер

Один сервер ещё можно настроить руками.



Один сервер
(физический или виртуальный)



Работает, но это плохо масштабируется.
Каждое новое окружение настраивается вручную, возможны расхождения.

Теперь: десятки сервисов и окружений

Приложение — это уже сложная система из множества компонентов.



Ответы индустрии



Containers

Приложение и его зависимости упаковываются вместе. «Работает у меня» становится ближе к «работает везде».



Orchestration

Управление множеством контейнеров: запуск, перезапуск, масштабирование, распределение по серверам. (например, Kubernetes)



Infrastructure as Code (IaC)

Вся инфраструктура описывается как код: сеть, серверы, хранилище, доступы, сервисы. (например, Terraform)



DevOps

Разработка и эксплуатация объединяются в единый процесс. Команда отвечает не только за код, но и за его доставку и работу в проде.

Как меняется подход

Было

- ✓ Настроили сервер руками
- ✓ Каждое окружение уникально
- ✓ Сложно повторить
- ✓ Масштабировать тяжело



Стало

- ✓ Опишите нужное состояние (код)
- ✓ Примените автоматически
- ✓ Получите одинаковое окружение
- ✓ Легче масштабировать

Новые вызовы



Сложность системы

Много сервисов, зависимостей и окружений.



Риски

Ошибочная конфигурация может автоматически распространиться на десятки систем.



Безопасность

Нужны автоматические проверки, управление доступом и секретами.

Примеры технологий



Docker
Контейнеризация



Kubernetes
Оркестрация



Terraform
Инфраструктура как код



AWS / GCP / Azure
Облачные платформы



Недостаточно один раз запустить систему.

Нужно уметь воспроизводимо собирать, разворачивать, масштабировать и восстанавливать её.

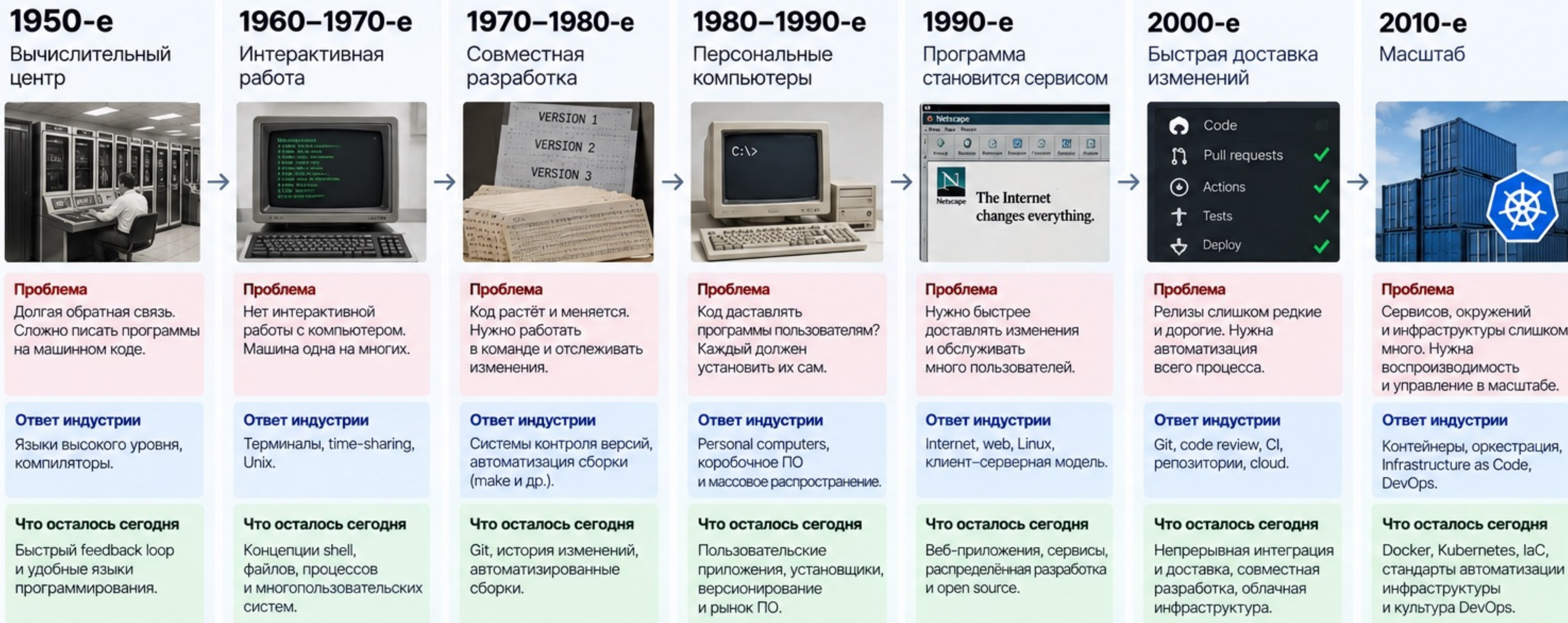
Автоматизация из удобства превращается в необходимость.

Чем больше возможностей у команды, тем важнее процессы, которые делают систему предсказуемой.

70 лет разработки в одной картинке

Каждая новая практика появлялась как ответ на новую проблему.

Разные эпохи.
Одна логика:
меньше ограничений —
больше возможностей.



Что усложнялось со временем?



Современная разработка — это не набор модных инструментов, а накопленные ответы на десятилетия инженерных проблем.

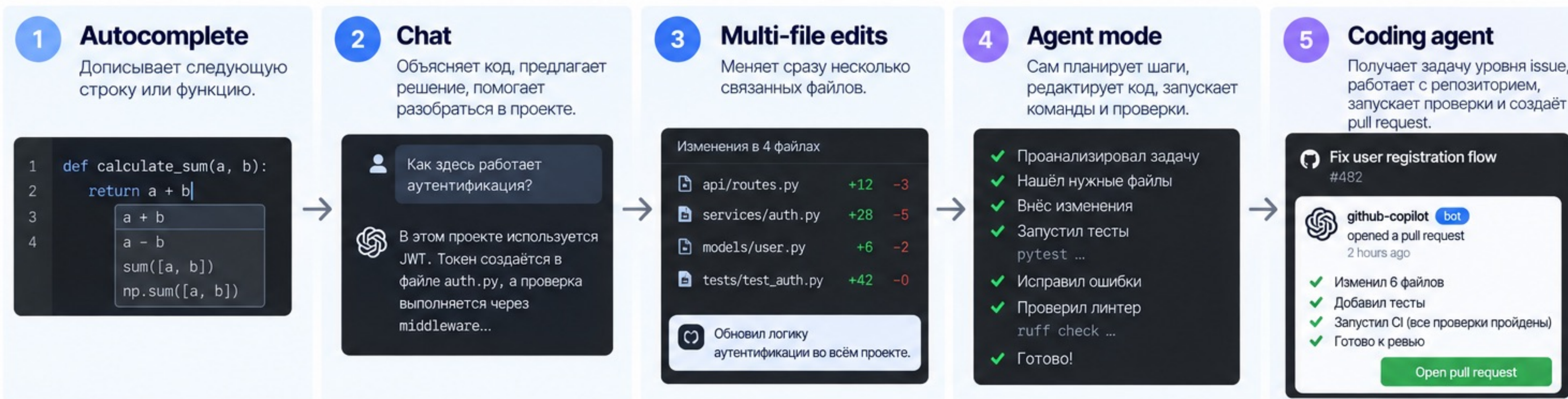
И история продолжается.
Следующий переломный момент — **AI**.

Мы находимся внутри следующего перехода

Same developers.
New superpowers.

2020-е — сегодня · AI начинает работать уже не только вокруг кода, но и с самим кодом

То, что ещё недавно было авто-дополнением, сегодня превращается в автономных агентов, которые могут работать с репозиторием, запускать инструменты и готовить pull request.



Как меняется роль разработчика



Что стало дешевле?

- ✓ Написать boilerplate
- ✓ Разобраться в незнакомом коде
- ✓ Сделать первый вариант реализации
- ✓ Рефакторить
- ✓ Написать тесты и документацию
- ✓ Выполнять типовые задачи

Что не исчезло?

- ✓ Постановка задачи и выбор подхода
- ✓ Архитектурные решения
- ✓ Понимание контекста и ограничений
- ✓ Проверка и валидация результата
- ✓ Ответственность за качество и безопасность
- ✓ Работа с людьми и продуктом



Предыдущие десятилетия автоматизировали процессы вокруг кода. Теперь автоматизация пришла к самой работе с кодом.

Меняется не только скорость разработки — меняется распределение работы между человеком и инструментом.

Мы не просто используем новые инструменты. Мы находимся внутри нового этапа в истории разработки.

Если код пишет AI — что остаётся инженеру?

Лучший код — тот, который решает правильную проблему.

Чем дешевле становится генерация кода, тем важнее становится качество решений вокруг него.

AI может ускорить реализацию, но не снимает необходимость понимать задачу, ограничения системы и последствия изменения.

Что всё чаще делает AI

Execution: быстро, много, без усталости



Генерирует boilerplate

Шаблонный код, типовые структуры, конфигурации.



Пишет функции и классы

На основе описания, примеров или существующего кода.



Делает refactoring

Упрощает, переносит, оптимизирует.



Создаёт tests

Unit, integration, e2e, фикстуры.



Объясняет и анализирует код

Помогает разобраться в незнакомом проекте.



Вносит изменения в несколько файлов

Учитывает зависимости и контекст проекта.



Запускает checks и готовит PR

Линтеры, тесты, анализ, описание изменений.



Задача

Цель, требования, контекст, ограничения



AI генерирует изменение

Предлагает решение, пишет и изменяет код



Инженер оценивает контекст

Понимает систему, проверяет корректность, учитывает риски



Review



Tests



Security



Architecture



Решение о релизе

Более надёжное и осознанное изменение в проде

За что всё ещё отвечает инженер

Judgment, context, ответственность



Понять, что нужно сделать

Проблема, цели, критерии успеха.



Выбрать архитектурный подход

С учётом долгосрочных последствий.



Учитывать ограничения системы

Производительность, надёжность, бюджет, команда, сроки.



Проверить корректность результата

Код компилируется — не значит, что он верный.



Оценить риски

Что может сломаться? Как откатить?



Обеспечить сопровождаемость

Читаемый код, понятная структура, документация.



Отвечать за безопасность

Уязвимости, данные, доступы, соответствие политикам.



Принять решение о релизе

И нести ответственность за последствия.

Что становится важнее



Context

Понимание системы, бизнеса и реальных ограничений.



Judgment

Выбор между несколькими приемлемыми решениями.



Verification

Умение доказать, что изменение действительно корректно.



Responsibility

Кто-то должен отвечать за последствия.



Ценность инженера смещается от производства кода к постановке задачи, проверке решений и ответственности за систему.

Чем проще получить код, тем важнее понимать, какой код вообще нужен и можно ли ему доверять.

AI может предложить решение. Инженер отвечает за то, что это решение подходит системе.

Жизнь одного изменения

От идеи до пользователя — и обратно

Даже небольшое изменение проходит через несколько этапов и ролей.
Чем зрелее команда, тем меньше из этих шагов остаётся полностью ручными.

Рабочий процесс — это не прямая, а цикл.

Пример задачи:

“Добавить уведомление о переносе занятия”



Что проходит через весь процесс



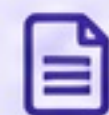
Качество

Тесты, код-ревью, метрики



Безопасность

SAST/DAST, управление доступом, секреты, аудит



Документация

Описания, changelog, инструкции



Автоматизация

CI/CD, инфраструктура как код, повторяемые процессы

В учебной задаче



Написал код



Запустил

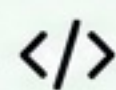


Готово

В продуктовой разработке



Понял задачу



Изменил код



Проверил (ревью, тесты)



Собрал версию



Доставил в прод



Наблюдаю за работой



Исправляю и улучшаю



Изменение ценится не тогда, когда код написан, а когда оно безопасно дошло до пользователя и продолжает работать.

Production — часть разработки, а не место, куда код «выкидывают» после завершения работы.

Хорошая разработка — это не один коммит, а непрерывный процесс улучшения продукта.

Кто вообще всем этим занимается?

Один lifecycle — много зон ответственности

Разработка — это командная работа. У каждого этапа есть свои участники, и одна роль может быть вовлечена в несколько этапов. Роль — не обязательно отдельный человек. В маленькой команде один человек может совмещать несколько ролей.

Хороший продукт — это результат совместной работы.



В маленькой компании

Один человек может совмещать несколько ролей.



Product

Analyst

Developer

QA

DevOps

Support

Отсутствие отдельной должности не означает отсутствие ответственности.

В большей компании

Роли разделяются, но всё равно тесно взаимодействуют.



PM



Аналитик



Разработчик



QA



DevOps



Security



Support

...

У каждого своя зона ответственности, но продукт — это общая задача.



Роли — это способ распределить ответственность, а не обязательный список должностей.

Чем сложнее продукт, тем важнее понимать, кто отвечает за каждый участок lifecycle.

Если ответственность нигде явно не закреплена, это не значит, что задача исчезла. Обычно это значит, что её никто не делает.

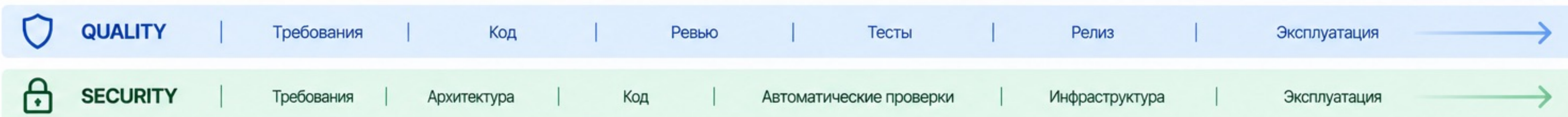
Процесс — не конвейер с движением только вправо

Проверки и обратная связь постоянно возвращают изменение на предыдущие этапы

В реальной разработке задача редко проходит все этапы с первого раза.

Чем раньше команда обнаружила проблему, тем дешевле её исправить. Поэтому хороший процесс специально создаёт много точек обратной связи.

Возвраты назад — это не неудача, а часть процесса.



Типичные примеры возвратов



«А что должно происходить в этом случае?»

Разработка → Требования



«Этот код слишком сложно поддерживать»

Ревью → Разработка



«Integration test упал»

Проверки → Разработка



«После релиза выросло количество ошибок»

Мониторинг → Разработка

Что делает automation?



- ✓ Одинаково повторяет проверки
- ✓ Быстро даёт обратную связь
- ✓ Не забывает обязательные шаги
- ✓ Останавливает плохое изменение до production
- ✗ Но не решает за команду, что правильно

Стоимость исправления ошибки



Чем позже найдена проблема, тем дороже её исправить.



Хороший инженерный процесс не устраняет возвраты назад — он делает их быстрыми, дешёвыми и управляемыми.

Чем раньше обнаружена проблема, тем меньше её стоимость.

Цель процесса — не пройти все этапы с первого раза, а быстро получать обратную связь и двигаться к правильному решению.

А GitHub завтра точно будет?

Доступность инструментов, vendor dependency и контроль над данными

Внешний сервис может работать сегодня, но инженерный процесс не должен полностью зависеть от того, что он будет доступен завтра. При выборе инструмента важно учитывать не только его возможности, но и доступность, лицензирование, оплату, хранение данных и возможность миграции.

Надёжность —
это не только
про uptime,
но и про контроль,

Команда зависит от внешней платформы

Примеры популярных сервисов



- Репозитории
- Issues и проекты
- CI/CD
- Артефакты и образы
- Зависимости (package registry)
- Секреты
- Интеграции
- История команды



Что произойдёт, если сервис станет недоступен?

Неважно, по какой причине: блокировки, сбой, изменение политики, проблемы с оплатой или другие факторы.

Возможные последствия



Разработка

Не получается git clone / push.

Команда не может вносить изменения.



CI/CD

Pipeline не запускается.

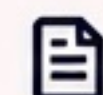
Нет новых сборок и проверок.



Зависимости

Не скачиваются пакеты или образы.

Сборка и разработка блокируются.



Артефакты

Недоступны предыдущие сборки.

Нельзя быстро откатиться.



История и контекст

Часть обсуждений и решений остаётся во внешней системе.

Теряется важный инженерный контекст.



Реальный контекст (2026)

В мае–июле 2026 года пользователи в РФ сообщали о проблемах с доступом к GitHub и отдельным его ресурсам; при этом Роскомнадзор заявлял, что GitHub не блокируется.

Для инженера важна не причина сбоя, а вопрос: продолжит ли команда работать, если внешний сервис временно исчезнет?



Что делает зрелая команда

Не отказывается от удобных инструментов, а убирает single point of failure.



Репозитории

Зеркало, backup, self-hosted вариант.



Артефакты и пакеты

Внутренний registry, кэширование, сохранение критичных зависимостей.



CI/CD

Возможность запускать ключевые pipeline внутри компании.



Данные

Регулярный export и backup, понятный план миграции.



Доступ

Не строить критичные процессы на одном внешнем аккаунте/сервисе.



Иногда это уже не выбор команды

Требования проекта, заказчика или регулятора могут ограничивать, где хранятся исходный код, артефакты и инфраструктура.

В ряде кейсов включения ПО в реестр российского ПО в 2026 году отдельно отмечалось хранение исходного кода на GitHub/GitLab.com/Azure DevOps и критические зависимости от внешних сервисов как проблемные факторы.



Хорошая инженерная система должна переживать недоступность отдельного внешнего сервиса.

Контроль над source code, artifacts, dependencies и данными — часть архитектуры разработки.

Удобные инструменты ускоряют работу, но устойчивость команды важнее, чем доступность конкретного сервиса.

Облако не всегда облако

Self-hosted инструменты: больше контроля — больше ответственности

Собственный GitLab, CI runners, registry и другие сервисы уменьшают зависимость от внешнего SaaS, но теперь команда сама отвечает за их доступность, обновление, безопасность и восстановление.

Инструменты разработки — тоже продукт, который нужно поддерживать.

SaaS (внешний сервис)

GitHub, GitLab.com, Cloud CI и другие



GitHub



GitLab.com



Azure DevOps



GitHub Actions



Docker Hub



и другие SaaS-инструменты

Провайдер отвечает за:

- ✓ Инфраструктуру
- ✓ Доступность платформы
- ✓ Обновления и патчи
- ✓ Масштабирование
- ✓ Часть backup / recovery
- ✓ Физическую безопасность

Команда в основном использует сервис и настраивает его под свои задачи.

Платим поставщику за эксплуатацию

Выбор зависит от контекста



Контроль



Зависимость от поставщика



Операционная нагрузка



Это не вопрос «что лучше», а вопрос баланса рисков, затрат и возможностей команды.

Self-hosted (своя инфраструктура)

Например, GitLab Self-Managed



А кто поддерживает теперь это всё?

Сервис больше не «где-то в интернете» — это наша инфраструктура, за которую отвечает команда.

Эксплуатация становится нашей инженерной задачей

Что появляется вместе с контролем



Обновления

- Регулярные обновления платформы и компонентов (GitLab, Runners и др.)
- Следим за совместимостью версий



Резервные копии

- Репозитории
- База данных
- Артефакты и registry
- Конфигурации и секреты



Восстановление

- Нужно уметь восстановиться
- Регулярно проверяем процедуру restore
- Совместимость версий и тесты восстановления



Масштабирование

- Свои CI runners и их ресурсы (CPU, RAM, storage)
- Планируем рост нагрузки
- Обеспечиваем отказоустойчивость



Безопасность

- Управление доступом
- Хранение токенов и секретов
- Обновления и патчи уязвимостей
- TLS, hardening, аудит
- Мониторинг и реагирование



Self-hosted не означает «бесплатно и независимо».

Это обмен зависимости от поставщика на ответственность за собственную инфраструктуру. Инструменты разработки сами становятся production-системой, которую нужно поддерживать.

«Если сервис критичен для разработки, он должен иметь такой же уровень надёжности, как и продукт, который мы создаём».

А интернета здесь нет

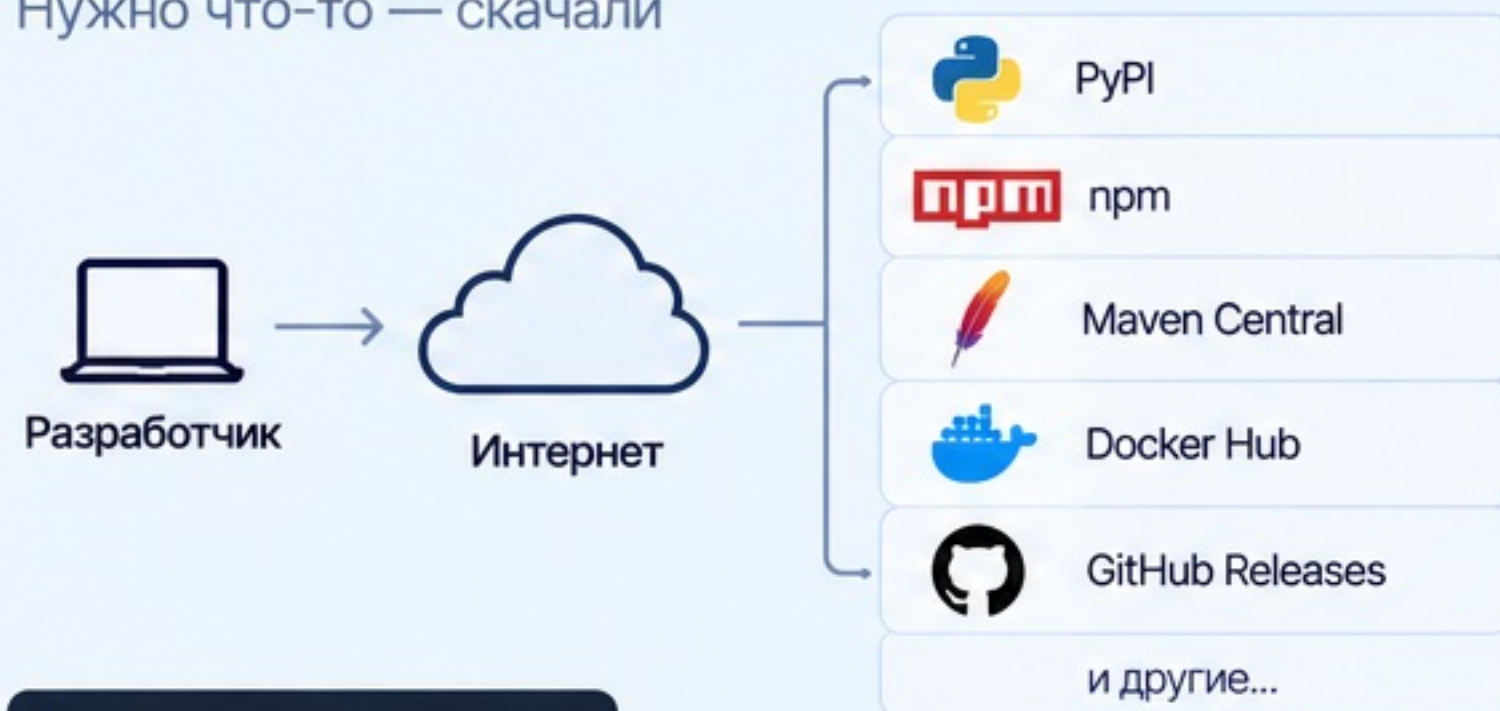
Закрытый контур и управление dependencies

В изолированной инфраструктуре: приложение не может во время сборки просто сходить в интернет за пакетами, образами или обновлениями. Всё необходимое нужно заранее получить, проверить и перенести внутрь.

В продакшене
нет интернета —
значит, интернет
должен быть в процессе,
а не в команде.

Обычная разработка

Нужно что-то — скачали



```
$ # Привычные команды
pip install requests
npm install react
mvn package
docker pull nginx
apt install curl
```

- ✓ Прямой доступ к внешним репозиториям
- ✓ Всегда актуальные версии
- ✓ Минимум ручных действий



Интернет
недоступен

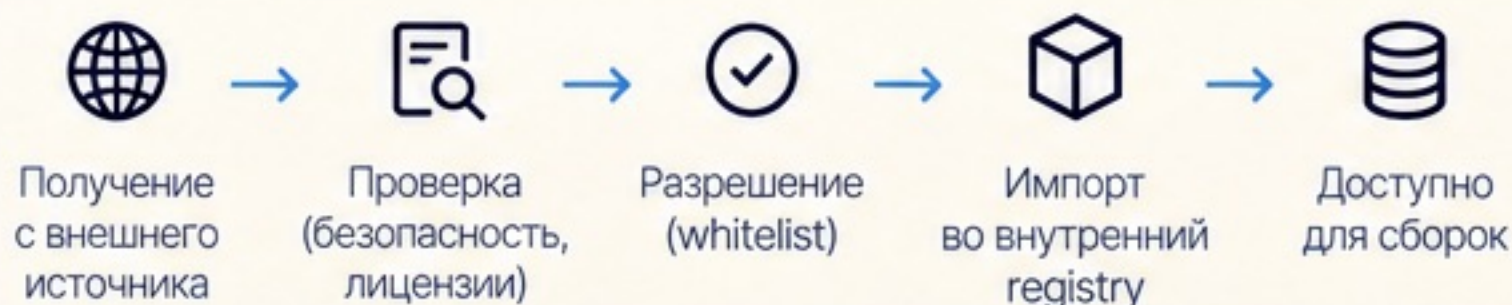
Сборка
должна работать
без доступа
к внешним
ресурсам

Закрытый контур

Всё необходимое — внутри инфраструктуры



Как зависимости попадают внутрь?



Кейс: Московская биржа



После закрытия прямого доступа к внешним репозиториям артефактов у разработчиков перестали работать привычные команды установки (npm install, pip install и др.), часть сборок остановилась. Сначала зависимости переносили вручную, затем процесс превратили в отдельный внутренний workflow с проверками, whitelist и автоматизацией.

Что нужно предусмотреть внутри?

- Package registry**
PyPI, npm, Maven и др. (проxy или внутреннее зеркало)
- Container registry**
Разрешённые OCI-образы
- Политики и whitelist**
Какие зависимости можно использовать
- Проверка безопасности**
CVE, malware, лицензии, provenance
- Обновления**
Процесс регулярного импорта новых версий



В закрытом контуре зависимость перестаёт быть просто строкой в конфигурации — она становится объектом поставки и контроля.

Воспроизводимая сборка требует, чтобы все необходимые артефакты были доступны внутри инфраструктуры.

Ограничения не отменяют инженерную культуру — они просто делают управление зависимостями частью процесса.

Работало на Windows. Теперь должно на Linux

Импортозамещение и совместимость — это не "поменять одну коробку на другую"

Замена ОС или СУБД почти всегда затрагивает приложение, библиотеки, инфраструктуру, инструменты, интеграции и процедуры миграции данных. Это комплексный проект, а не простая перестановка компонентов.

Другая ОС —
это не только другой
интерфейс, а другая
экосистема.

Было

Windows Server + MS SQL Server

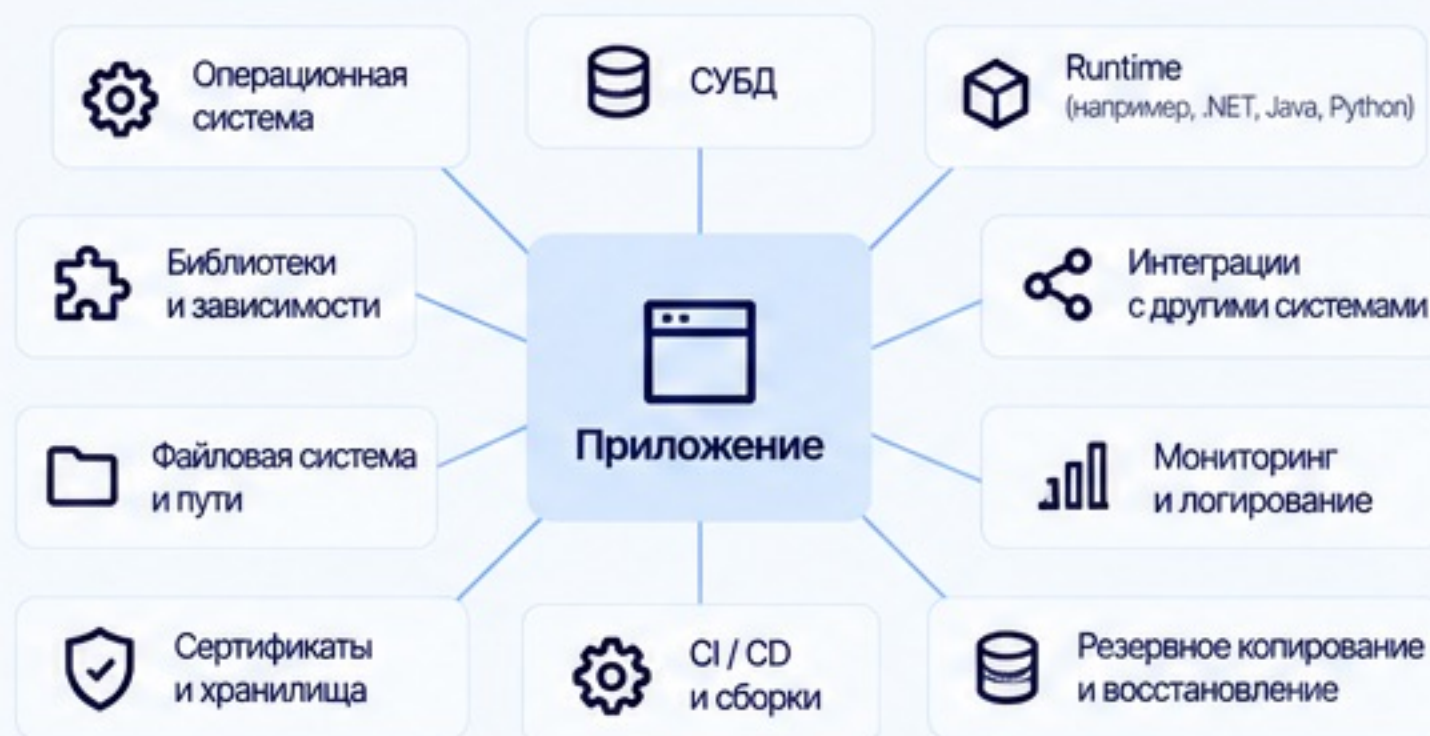


Windows Server 2016

.NET / IIS
MS SQL Server 2016
PowerShell / .bat
Windows Services
Active Directory
Брендовые инструменты

Приложение и его зависимости

Меняется вся экосистема, а не только ОС



Что из этого действительно совместимо?
Где потребуются изменения?

Стало

Linux + PostgreSQL



Linux (Ubuntu, Astra Linux и др.)

PostgreSQL
.NET / Java / Python (Linux)
Bash / sh
systemd
LDAP / FreeIPA (вместо AD)
Открытые инструменты

Что ломается неожиданно?



SQL-диалект
Различия в синтаксисе и типах данных



Пути и файловая система
C:\... больше не существует, другие права и структура



Сервисы
Windows Services ≠ systemd



Скрипты
PowerShell / .bat требуют переписывания



Сертификаты
Форматы (.pfx), хранилища и механизмы работы отличаются



Мониторинг и backup
Привязка к специфичным инструментам Windows



Сторонние интеграции
Драйверы, SDK, агенты могут быть недоступны

Кейс: Haulmont / СЭД ТЕЗИС



Миграция с Windows + MS SQL на Linux + PostgreSQL

Система обслуживала около 2 000 пользователей, база данных — ~700 ГБ.

Миграция потребовала не только переноса данных, но и адаптации работы приложения, выбора новой инфраструктуры, тестирования и планирования окна миграции. Перенос production БД объёмом 700 ГБ выполнили за выходные, фактическая миграция заняла 13 часов.

2 000

пользователей

700 ГБ

объём БД

13 часов

время миграции

Как выглядит миграция на практике

Это не одна команда, а последовательный процесс



700 GB DB ≠ просто pg_dump && restore

Нужно учитывать производительность, совместимость, время простоя и валидацию данных.

Другой пример: комплексная замена



Россельхозбанк (проект)

В одном из проектов одновременно меняли Windows на Astra Linux, Oracle на PostgreSQL, middleware и архитектуру приложения. Авторы отмечают высокий риск из-за одновременной замены связанных технологий.

Чем больше компонентов меняем одновременно, тем сложнее понять источник проблемы.



Платформа — часть архитектуры приложения. Её замена почти всегда становится полноценным инженерным проектом.

Совместимость нужно проверять не только у кода, но у всей цепочки: build → runtime → data → integrations → operations.

Успешная миграция — это не «чтобы заработало», а «чтобы стабильно работало и поддерживалось».

Заказчик тоже участвует в архитектуре

Технологическое решение существует внутри требований и ограничений проекта

В реальной разработке архитектуру определяет не только команда, но и требования заказчика: инфраструктура, безопасность, интеграции, эксплуатация, бюджет и нормативные требования. Задача инженера — найти оптимальное решение в этих рамках.

Лучшая архитектура — это не та, что нам нравится, а та, которая работает в реальности.

Теоретически

Много вариантов



~ 20+

возможных технологий
и подходов

Реальность: архитектура под ограничениями

Требования заказчика формируют границы допустимых решений



Инфраструктура

- Только on-prem (без облаков)
- VMware, определённые версии
- Конкретные ОС (например, Astra Linux)
- Использование существующего оборудования
- Ограничения по сети и доступу



Безопасность и соответствие

- Требования ИБ и регуляторов
- Аутентификация (AD / LDAP)
- Шифрование и управление ключами
- Журналирование и аудит
- Допустимые компоненты и версии
- Проверка на уязвимости
- Соответствие ГОСТ / внутренним политикам



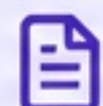
Эксплуатация

- Кто будет поддерживать
- Требования к отказоустойчивости (SLA)
- Резервное копирование
- Мониторинг (существующие инструменты)
- Регламенты обновлений
- Ограничения по времени работ (maintenance window)



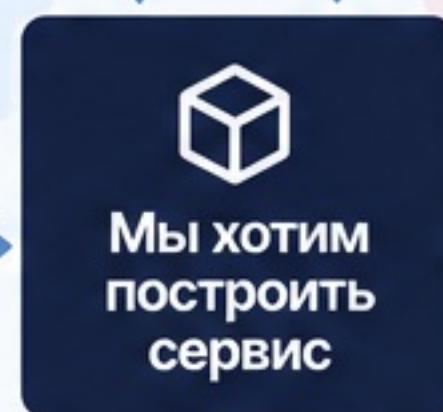
Интеграции

- Работа с существующими системами
- Корпоративная шина / ESB
- Базы данных заказчика
- Legacy API и форматы данных
- Почта, СЭД, ERP и другие системы
- Требования к протоколам и форматам



Приёмка и документация

- Техническое задание и критерии приёмки
- Испытания и подтверждение соответствия
- Документация (архитектура, инструкции, регламенты)
- Требования к передаче исходного кода
- Сопровождение и обучение



Требование ≠ прихоть

Ограничения заказчика обычно связаны с реальными причинами: существующей инфраструктурой, безопасностью, стоимостью, лицензиями, регуляторными требованиями или необходимостью интеграции.



Задача инженера

- Понять ограничения и их причины
- Предложить лучшее решение в рамках доступных вариантов
- Оценить последствия и риски
- Понятно объяснить компромиссы заказчику
- Обеспечить реализуемость и поддержку в эксплуатации



Главная мысль

Архитектура — это не выбор “самой лучшей технологии”, а поиск лучшего решения внутри реальных ограничений.

Хороший инженер сначала выясняет требования, а потом выбирает стек.

В результате

Немного допустимых вариантов

Пример: выбор базы данных

✓	PostgreSQL	Подходит (утверждена)
✓	Oracle	Подходит (уже используется)
✗	MongoDB	Не соответствует требованиям
✗	Cloud DB	Запрещено (нет облаков)
✗	SQLite	Не подходит для отказоустойчивости

Пример: инфраструктура

✓	VMware + Linux	Допустимо
✓	Физические серверы	Допустимо
✗	Kubernetes	Не используется (нет компетенций)
✗	AWS / Yandex Cloud	Запрещено
✗	Managed DB	Запрещено

2–3

реально допустимых варианта
вместо десятков

Production может находиться вообще не у вас

Поставка ПО в инфраструктуру заказчика

В большинстве enterprise-проектов продукт устанавливается и работает в инфраструктуре заказчика. Команда передает не только код, но и полный комплект для установки, а дальше среда уже не под её полным контролем.

Хороший продукт работает в любой среде. Но ещё лучше — когда понятно, как его туда доставить.

Наш SaaS (привычный сценарий)

Мы разворачиваем и управляем сами



- ✓ Инфраструктура под нашим контролем
- ✓ Можем быстро развернуть новую версию
- ✓ Есть прямой доступ к логам и мониторингу
- ✓ Контролируем обновления и откаты
- ✓ Оперативно реагируем на инциденты

У заказчика (on-prem / закрытый контур)

Мы поставляем продукт, а дальше он живёт в их инфраструктуре



⚠ После передачи продукта среда уже не под полным контролем команды. Установка, конфигурация, доступы и эксплуатация — на стороне заказчика.

Что усложняется?

- Доступ к инфраструктуре**
Нет прямого SSH, VPN или доступа к логам
- Разные окружения**
OS, версии ПО, сеть, проху, сертификаты, СУБД
- Долгие циклы обновлений**
Только по согласованию, часто в maintenance window
- Безопасность и регламенты**
Дополнительные проверки и согласования
- Коммуникация**
Нужно взаимодействовать с командой заказчика (Админы, ИБ, эксплуатация и др.)

Что входит в поставку?

Release — это не просто бинарник, а полный комплект для установки и сопровождения

Приложение
Пакет / image / installer

Зависимости
Все необходимые библиотеки, образы, пакеты

Конфигурация
Шаблоны, параметры, переменные

Миграции
Изменения БД и данные

Документация
Инструкция по установке и эксплуатации

Проверки
Чек-листы, тесты, критерии приёмки

Rollback
План отката и рекомендации

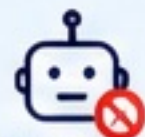
Кейс: поставка в закрытый контур

Государственная организация (типовой сценарий)

Продукт разворачивается в полностью изолированной инфраструктуре заказчика. Команда передаёт установочный пакет, все зависимости, инструкции и скрипты миграции. Установка выполняется специалистами заказчика по согласованному регламенту. Обновления — только после прохождения дополнительных проверок ИБ.

Успешный релиз — это не момент, когда собрался артефакт, а момент, когда он запущен у заказчика.

А что с AI?



В ряде проектов передача кода, данных или даже описаний системы во внешние AI-сервисы (например, ChatGPT, Claude и др.) может быть запрещена политиками заказчика.

Всегда нужно учитывать, где хранятся код, данные и запросы, и соответствуют ли инструменты требованиям проекта.



Если production находится у заказчика, release заканчивается не сборкой, а успешной установкой и проверкой в реальной среде.

Чем меньше у команды прямого контроля над production, тем важнее воспроизводимость поставки, диагностика и документация.

«Хороший релиз — это не тот, который мы задеплоили, а тот, который работает у заказчика».

Три человека могут позволить себе хаос

Маленькая команда оптимизирует не процесс, а скорость поиска работающего продукта

Когда людей мало, коммуникация дешева, роли совмещаются, а многие проблемы можно решить вручную. Поэтому формальные процессы часто появляются позже.

23
На ранней стадии
главное — делать,
а не документировать.
(но помнить
о последствиях)

Команда из трёх человек

Много ролей, короткие коммуникации, быстрые решения



Product + Analyst + Support

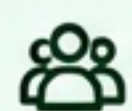
Понимание продукта,
общение с пользователями,
приоритизация, поддержка

Backend + DevOps

Разработка,
инфраструктура,
деплой, мониторинг
(иногда)

Frontend + QA

Разработка интерфейса,
тестирование,
исправление багов,
релизы



3
человека



1
сервер



~ 20
пользователей



Ручные решения
работают. Скорость
важнее формальностей.

Что получается хорошо



Быстрая коммуникация

Всё обсуждается сразу, без встреч
и бюрократии



Простые решения

Архитектуру можно поменять за день



Быстрые релизы

Один человек собирает и выкатывает



Гибкость

Легко пробовать новое и pivot'ить



Высокая ответственность

Все вовлечены и понимают продукт

Но есть и обратная сторона



Нет документации

«Мы и так знаем, как всё работает»



Общие доступы

У всех почти одинаковые права



Бэкапы без проверки

Бэкап есть, но никто не проверял restore



Безопасность откладывается

Фичи важнее, чем hardening



Мониторинг минимальный

Проблемы часто узнаем от пользователей



Знания в головах

Критичные вещи знает один человек
(Bus factor = 1)

Что может пойти не так?



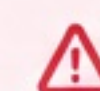
Что, если DevOps-разработчик заболел?

Кто будет деплоить и разбираться с инфраструктурой?



Что, если случайно удалили данные?

Есть ли проверенный бэкап и план восстановления?



Что, если появился четвёртый разработчик?

Сможет ли он быстро войти в контекст?



Что, если случится серьёзный инцидент по безопасности?

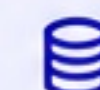
Кто будет разбираться и как быстро?

Что приводит к изменениям?



Команда растёт

Уже невозможно держать всё в голове



Появляются реальные клиенты

Простой начинает стоить денег



Растёт сложность продукта

Больше компонентов и интеграций



Требования регуляторов или enterprise-заказчика

Появляются обязательные процессы



Происходит серьёзный инцидент

Внезапно понимаем, зачем нужны мониторинг,
бэкапы и управление доступами



3 человека — иллюстрация, а не граница.

Размер команды — только один из факторов.
Гораздо важнее: тип продукта, данные, отрасль,
требования заказчика и цена ошибки.



Маленькая команда может долго компенсировать отсутствие процессов высокой скоростью коммуникации и личной ответственностью.

Проблемы начинаются, когда сложность продукта растёт быстрее, чем способность людей держать всё в голове.

Хаос — нормален на старте.
Но он не масштабируется.

Когда процессы начинают окупаться

Процессы появляются не из любви к бюрократии, а когда хаос становится дороже

Чем больше людей, пользователей, данных и инфраструктуры, тем дороже становится ручное управление.

Больше людей —
больше возможностей.
Но ещё больше
ответственности.

